

1) Objetivos:

- Compreender os operadores de Processamento Morfológico de Imagens
- Utilizar funções da biblioteca “*morph.py*”
- Manipular imagens binárias .
- Gravar imagens e vídeos .
- Elaborar o relatório em equipe.

2) Pré-requisitos: Jupyter Notebook, Python 3.x, NumPy, OpenCV e Matplotlib.

- (a) Utilize suas próprias imagens obtidas no Lab3 - imagem dos membros juntos do grupo e imagem dos avatar juntos, para o relatório. Adicionalmente utilize imagens relacionadas ao projeto final da equipe.
- (b) Dentro do ambiente virtual criado previamente (PDI26), crie uma pasta do “Lab7”, e nela baixe o arquivo “*morph.py*” com o seguinte comando:
`wget https://raw.githubusercontent.com/fzampirolli/morph/main/morph.py`
- (c) No MOODLE: baixe os arquivos fornecidos “*pdi26_lab7_morph.ipynb*” e “*water_coins.png*”, “*noisy_fingerprint.png*”, “*text_gaps.png*” na sua pasta do “Lab7”.
(Obs. ao final da aula, apague todos arquivos do computador.)

3) Atividades Práticas Experimentais:

Abra o arquivo “**pdi26_lab7_morph.ipynb**”, no Jupyter Notebook.

Estude detalhadamente todas as funções fornecidas e os códigos.

Elabore seu código de programa para cada realizar cada uma das atividades propostas.

- (3.a). Elabore um programa para realizar as operações de EROÇÃO, DILATAÇÃO, ABERTURA, e FECHAMENTO, sobre as imagens em grupo dos integrantes da equipe bem como nos avatares, e apresente os resultados. Elabore uma análise detalhada de cada imagem, experimentado diversos tamanhos (3x3 até 21x21) de Elemento Estruturante – KERNEL - e compare-as entre si, quanto ao aspecto visual (filtragem, aparência, representação, etc) da imagens resultantes. Lembre-se de binarizar as imagens antes da aplicação dos operadores. Assim, dentro desta análise, discuta se a binarização influenciou no resultado e qual método foi mais interessante para sua aplicação.

ELABORAÇÃO DE PROGRAMAS COM A WEBCAM:

- (3.b). Elabore um programa para realizar continuamente sobre a imagem da WEBCAM as operações de Extração de Fronteiras (Gradiente Morfológico), mostrando na tela em tempo real lado-a-lado a imagem original e a imagem extraída, permitindo ao operador escolher – via teclado – a extração com diversos tamanhos de kernel. Grave um vídeo demonstrativo

com os integrantes da equipe utilizando este programa com a webcam. Elabore uma análise detalhada e explique os artefatos ocorridos.

Obs. O comando `cv2.hconcat()` une duas ou mais imagens horizontalmente (lado a lado), desde que tenham a mesma altura. Ex.: `resultado = cv2.hconcat([img1, img2])`

4) Exercícios Propostos

(4.a). Elabore um programa para aplicar os operadores morfológicos imagens do seu trabalho final, explicando o objetivo deste procedimento dentro do contexto da aplicação.

5) **Questões de Análise do Processamento Morfológico de imagens:** elabore as respostas para cada uma das questões abaixo em forma de texto e/ou figuras, e apresente na parte de **Análise e Discussão** do relatório.

(5.a). Erosão e Estrutura: Ao aplicar uma erosão em uma imagem binária que contém ruídos brancos (pontos isolados de 1 pixel) utilizando um kernel 3x3 de *uns*, o que acontece com esses ruídos? Explique o porquê baseando-se no funcionamento do operador.

(5.b). Dilatação e Conectividade: Suponha que você tenha uma imagem de um texto digitalizado onde as letras estão levemente "quebradas" ou fragmentadas. Qual o efeito de aplicar uma dilatação com um elemento estruturante pequeno (ex: 3x3)? Existe algum efeito colateral indesejado se o kernel for grande demais?

(5.c). Extração de Contornos (Gradiente): A extração de fronteira pode ser feita subtraindo a imagem erosionada da imagem original ($A - (A \ominus B)$).

-- Prática: O que mudaria no contorno resultante se aumentássemos o tamanho do elemento estruturante **B** de 3x3 para 11x11?

(5.d). Abertura vs. Fechamento:

* -- Qual dessas operações você utilizaria para remover pequenos objetos indesejados (como poeira em um scan) sem alterar significativamente o tamanho dos objetos principais?

* -- Qual você utilizaria para fechar pequenos buracos ou fendas dentro de um objeto sólido?

(5.e). Análise de Kernel (Geometria): No código da Parte 2, definimos kernels retangulares, elípticos e em cruz.

* -- Desafio Prático: Se você aplicar uma dilatação em um único pixel branco no centro de uma imagem preta usando um "kernel_cross" (cruz), qual será o formato do objeto resultante? E se usar o "kernel_rect"?

- 6) **Relatório:** Elaborar o relatório em formato do JUPYTER NOTEBOOK - extensão .IPYNB, e hospedar no github, conforme instruções em aulas anteriores.

O relatório deverá conter pelo menos os seguintes tipos de Seções:

- Título do relatório
- Nome completo dos autores do relatório
- Data de realização dos experimentos
- Data de publicação do relatório
- Introdução – apresentando o que será descrito e relatado, bem como uma breve introdução ao assunto
- Procedimentos experimentais – explicando como realizar e executar as atividades
- Análise e discussão dos estudos realizados
- Conclusões
- Referências consultadas e indicadas.

Cada relatório deverá ser colocado numa pasta separada, junto com os arquivos pertinentes.

Opcional: uma página HTML da equipe contendo um índice com um link para cada relatório dos laboratórios.

ATENÇÃO: o RELATÓRIO deve ser de caráter didático, sendo autosuficiente para que um aluno consiga reproduzir os mesmos experimentos que a equipe realizou.

-X-X-X-

Referências:

- Rafael C. Gonzalez . **Digital Image Processing**, Rafael C. Gonzalez . 4.ed. Pearson, 2018. Chapter 9 “Morphological Image Processing”.
- Tutorial OpenCV e Python: *Image Thresholding*
https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html
- *Image Thresholding in OpenCV*. Satya Mallick, February 24, 2015:
<https://learnopencv.com/opencv-threshold-python-cpp/>

-X-X-X-